

ECE 157C / ECE 272C

Homework Assignment 4

Building an Orchestrated AI Analytics System

University of California, Santa Barbara
Instructor: Seoyeon Kim

Due: May 31, 2026 at 11:59 PM

1 Overview

In Homework 1, you built a code generation and execution agent capable of:

- generating Python code,
- executing the generated code,
- evaluating execution results,
- generating a final answer.

In Homework 2, you extended this system into an interactive analytics environment with:

- frontend/backend integration,
- conversational memory,
- visualization,
- multi-turn interaction.

In Homework 3, you built a plan-and-execute agent that:

- separates reasoning from execution,
- generates structured workflows,
- performs deterministic execution,
- exposes execution traces.

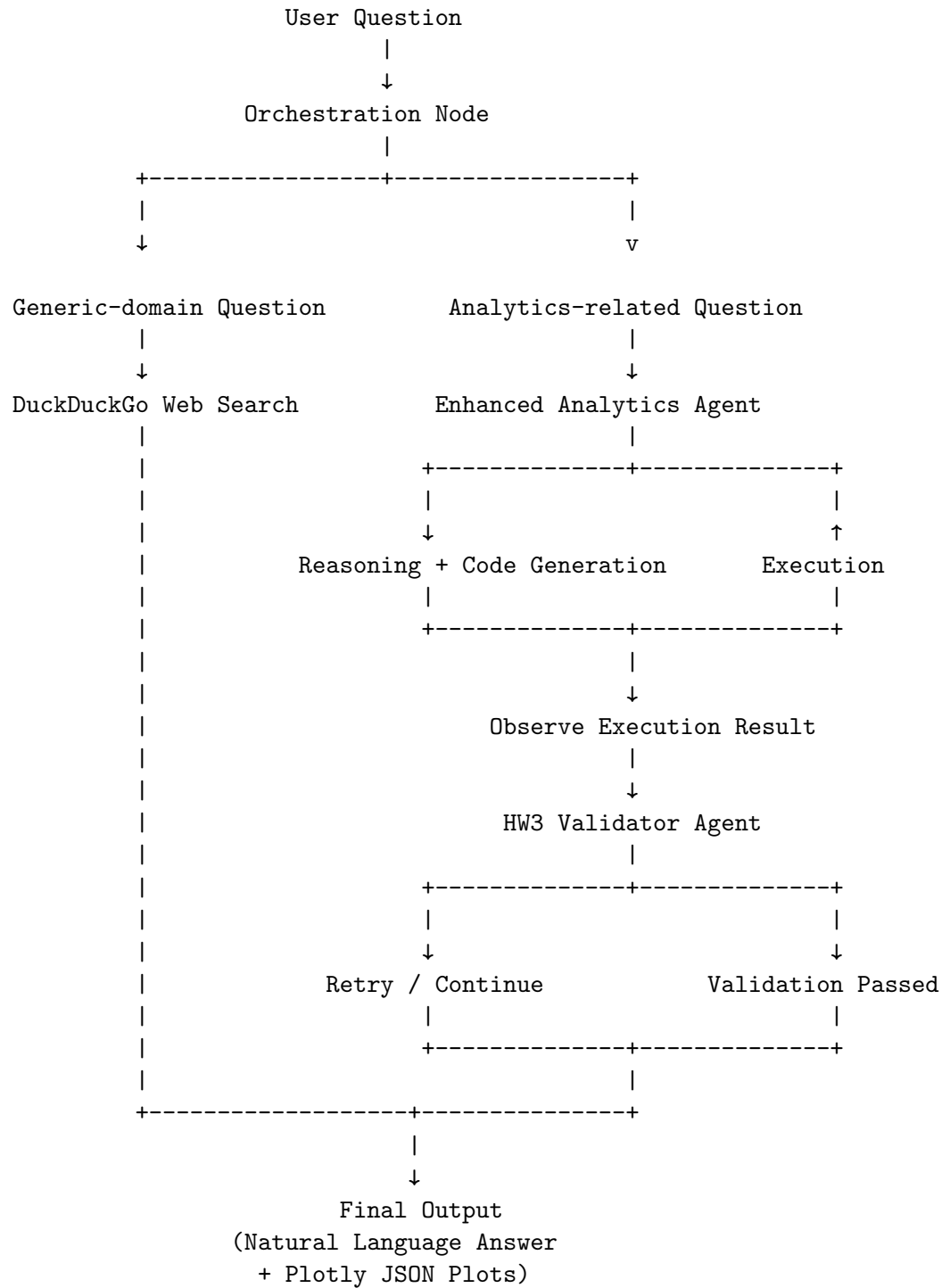
In this homework, you will extend your previous agents into a larger orchestrated AI analytics system that supports iterative reasoning/execution, tool usage, validation, web-grounded reasoning, and coordination between multiple agent components. The goal is not only to build an AI agent, but also to use the system to perform realistic analytics workflows, investigate findings through iterative interaction, and summarize the overall results in a presentation-style format.

2 Core Architecture

Your HW4 system should extend the agents you built in previous assignments.

2.1 High-Level Workflow

Your system should approximately follow the workflow below:



3 Orchestration Node

You must implement an orchestration/supervisor node.

Responsibilities:

- determine whether the user question is:
 - analytics-related,
 - or generic-domain,
- decide which tools or agents should be invoked,
- route outputs between components,
- decide whether additional reasoning/execution is needed based on validator feedback.

Example:

- If the question is:

“What caused the 2016 oil-price crash?”

the orchestration node may invoke DuckDuckGo web search.
- If the question is:

“Find undervalued technology companies from the dataset.”

the orchestration node may invoke: the analytics agent, the validator.
- If the validator determines that the analysis is incomplete or suspicious, the orchestration node may trigger another round of reasoning/code generation/execution.

4 Generic-Domain Questions

For generic-domain questions, your system must use DuckDuckGo search to retrieve relevant information, generate grounded answers, and provide citations or URLs.

You may use the following LangChain DuckDuckGo integration:

<https://docs.langchain.com/oss/python/integrations/tools/ddg>

Example questions:

- “What is retrieval-augmented generation?”
- “What caused the 2016 oil-price crash?”
- “What is NVIDIA’s primary business segment?”

5 Enhanced Analytics Agent

You must extend your HW1/HW2 analytics agent into a more autonomous and iterative analytics system.

Unlike HW1, your system should not simply receive predefined schema information, generate code once, execute it, and terminate.

Instead, the analytics agent should initially receive only the dataset file paths, and the user question. The agent should then autonomously inspect datasets, explore dataset structure, determine what analysis is needed, generate and execute analysis code, observe execution outputs, and decide whether additional reasoning or analysis is required before continuing or stopping.

Unlike one-shot code-generation systems, the execution environment should maintain persistent state across iterations. Variables, imported libraries, generated dataframes, intermediate outputs, and execution results may be reused in future reasoning/execution steps.

The system should behave similarly to a lightweight analytics sandbox or code-interpreter environment.

A lightweight local sandbox may be implemented by repeatedly executing generated Python code within a persistent execution namespace.

5.1 Stopping Condition

Your system should determine whether additional reasoning/execution is needed or whether the analysis is sufficiently complete. This decision may depend on execution outputs, reasoning state, and the overall completeness of the analysis.

6 Analytics Agent Output

After the reasoning/code-generation/execution process is completed and validated, the analytics agent should generate a structured output containing:

- the final natural-language answer,
- generated Plotly JSON visualizations,
- and optional execution summaries or reasoning traces.

The final output format may look similar to:

```
{
  "final_answer": "...",
  "plots": [
    {
      "title": "...",
      "plotly_json": {...}
    }
  ]
}
```

7 HW3 Validator Agent

You must reuse your HW3 plan-and-execute agent as a validator/checker agent.

Since HW3 primarily assumed single-table analytics workflows, you should extend and improve the validator agent to support multi-table operations and more complex analytics scenarios. This may require adding additional operators, reasoning capabilities, or validation logic for operations such as joins, unions, table alignment, and cross-table aggregation.

The validator agent should independently analyze the question and inspect the execution outputs, generated analytics results, numerical consistency, completeness of analysis, and whether the original question was sufficiently answered.

After that, the validator agent should compare its own reasoning/results with the outputs generated by the analytics agent. Based on this comparison, the validator agent may approve the analysis, request additional reasoning/execution, detect suspicious or inconsistent results, identify missing analysis, or recommend retry.

Important:

- validation is NOT simply checking whether code executed successfully,
- the validator should reason about the correctness and completeness of the analysis,
- the validator agent should act as an independent reasoning/checking component rather than a simple execution monitor.

8 Dataset

You will use the following Kaggle dataset:

[200+ Financial Indicators of US Stocks \(2014–2018\)](#)

Before designing analytics questions or workflows, first explore and understand the dataset structure, available features, data distributions, and overall dataset characteristics.

The dataset consists of separate CSV files for different years between 2014 and 2018. To encourage multi-table reasoning and temporal analysis, you must download and use at least two different yearly CSV files in your system.

This dataset contains company financial indicators, sector information, profitability metrics, valuation metrics, growth indicators, and future return-related information. The dataset supports multi-dimensional analysis, anomaly detection, comparative analytics, and iterative reasoning.

9 Reusing HW2 Components

Reuse and extend the chatbot-style frontend and conversational memory developed in Homework 2 to support interactive multi-turn analytics workflows.

10 Presentation and Report Requirements

Your report should include user-interface screenshots and text-based intermediate agent traces for at least 6 representative examples. Among these examples, include at least 2 generic-domain questions using DuckDuckGo web search, at least 2 analytics-oriented examples involving iterative

reasoning/code-generation/execution, and at least 2 examples where the validator agent requests retry or detects suspicious/incomplete analysis.

For analytics-oriented examples, create a realistic analysis scenario and interact with your system through multiple questions and follow-up questions. There should be at least 5 questions/interactions used within the scenario.

At least part of the scenario should involve cross-year or temporal analysis using multiple yearly CSV files from the dataset. The interaction should demonstrate how the system handles comparisons, trends, or changes across different years.

Based on the findings from the interaction, create a PowerPoint presentation (.pptx) summarizing the overall analysis results. The presentation should focus on high-level insights, conclusions, visualizations, and analysis summaries rather than simply listing the original user questions. Submit the generated .pptx file together with your report.

Your report should clearly explain the overall system design, including the orchestration node design, analytics agent design, validator agent design, execution/reasoning loop, and overall agent workflow. Include architecture/agent-design diagrams illustrating how different components interact with each other.

For each major node/component, explain its role, how it operates, what inputs/outputs it receives, and how prompts are designed or structured.

For the scenarios used in your presentation, choose 2 questions from the scenario along with the corresponding final answers, generated visualizations/plots, and the iterative reasoning/execution process used by your system. Explain how many reasoning/execution iterations were performed, what actions were taken at each step, what observations were made from execution results, and how those observations influenced the next reasoning or analysis step. Each analytics-oriented example should include at least 3 reasoning/execution iterations. These examples are separate from the 6 examples described above.

Your report should also briefly discuss stopping conditions, retry behavior, validation strategies, limitations/failure cases, lessons learned, implementation challenges, and possible future improvements.

11 Suggested Project Structure

Feel free to organize differently as long as the required functionality is supported.

```
project/
|
|-- orchestrator.py
|-- analytics_agent.py
|-- validator_agent.py
|-- memory.py
|-- web_search.py
|-- frontend/
|-- backend/
|-- datasets/
|-- documents/
|-- Report.pdf
|-- Presentation.pptx
'-- README.md
```

12 Grading

- Orchestration quality: 10%
- Iterative analytics loop: 20%
- HW3 validator integration: 20%
- Multi-agent/system design: 20%
- Report + Presentation: 30%